

Part 4: Client Proposal

What I Built

I developed the backend foundation for a matchmaking platform using NestJS and MongoDB. The system includes a personalized recommendation engine, swipe-based interactions (like/pass), mutual match detection, and secure JWT-based authentication with tokens stored in HTTP-only cookies for safer client-side session handling.

The matchmaking engine intelligently ranks profiles using weighted compatibility scoring based on shared interests, mutual age preferences, and location similarity while also preventing duplicate profile recommendations through swipe history tracking.

Enhancement

One of the major enhancements implemented was a custom compatibility scoring algorithm combined with a secure authentication architecture. Instead of displaying profiles randomly, the platform calculates compatibility using weighted scoring logic that prioritizes shared interests through Jaccard similarity, mutual age preference matching, and location relevance. This creates a more personalized and meaningful recommendation system for users.

For authentication, the platform uses JWT-based authentication with tokens stored securely in HTTP-only cookies. This approach reduces client-side token exposure, protects against XSS attacks, and ensures credentials are never accessible via JavaScript. The JWT payload contains only the user ID, keeping tokens lightweight while the HTTP-only cookie handles secure transport on every request automatically.

Next Steps

- Add real-time chat and notifications using WebSockets
- Write comprehensive unit, integration, and end-to-end test cases for authentication and matchmaking flows
- Perform load and stress testing to evaluate scalability and request handling under high traffic
- Implement profile image uploads using Amazon S3 for scalable and secure media storage

- Implement a reporting and moderation system with admin roles, dashboards, and user ban capabilities for platform safety
-

Technical Approach

The platform follows a modular backend architecture built with NestJS and MongoDB. User authentication is handled using JWT tokens stored in secure HTTP-only cookies, with a custom Auth Guard that verifies and decodes the token on every protected request. Match actions such as likes and passes are tracked separately, allowing the recommendation engine to exclude previously viewed profiles and dynamically calculate compatibility scores. Mutual likes trigger match creation logic that can later support messaging and real-time interactions.

Simple Architecture Diagram:

Client Application



HTTP-only Cookie (JWT Token)



NestJS API Server



Auth Guard (JWT Verification + Decode)



MongoDB Database



Recommendation Engine



Compatibility Scoring + Match Actions



Mutual Match Detection



Admin Dashboard & Moderation System